```
import Mathlib

/--
Determine all real numbers $\alpha$ such that, for every positive integer $n$, the integer

$$\lfloor \alpha \rfloor + \lfloor 2\alpha \rfloor + \cdots + \lfloor n\alpha \rfloor$$

is a multiple of $n$.

(Note that $\lfloor z \rfloor$ denotes the greatest integer less than or equal to $z$. For example, $\lfloor -\pi \rfloor = -4$ and $\lfloor 2 \rfloor = \lfloor 2.9 \rfloor = 2$.)
-/
theorem imo_2024_p1 :
   {(α : ℝ) | ∀ (n : ℕ), 0 < n → (n : ℤ) ∣ (∑ i in Finset.Icc 1 n, ⌊i * α⌋)}
   = {α : ℝ | ∃ k : ℤ, Even k ∧ α = k} := by
  rw [(Set.Subset.antisymm_iff ), (Set.subset_def), ]
  existsλx L=>(L 2 two_pos).rec λl Y=>?_
  useλy . x=>y.rec λS p=>?_
  · simp_all[λL:ℕ=>(by norm_num[Int.floor_eq_iff]:L(L:ℝ)*S⌊=L* S )]
    rw[p.2,Int.dvd_iff_emod_eq_zero,Nat.lt_iff_add_one_le,←Finset.sum_mul,←Nat.cast_sum, S.even_iff,
      ↪ ←Nat.Ico_succ_right,@ .((( Finset.sum_Ico_eq_sum_range))),Finset.sum_add_distrib ]at*
    simp_all[Finset.sum_range_id]
    exact dvd_trans ⟨2+((_:ℕ)-1),by linarith[((‹ℕ›:Int)*(‹Nat›-1)).ediv_mul_cancel$ Int.prime_two.dvd_mul.2<|by ·omega]⟩
      ↪ ↑↑(mul_dvd_mul_left @_ (p))
  suffices : ∀ (n : ℕ),⌊(n+1)*x⌋ =⌊ x⌋+2 * ↑ (n : ℕ) * (⌊-(⌊x)⌋))
  · zify[mul_comm,Int.floor_eq_iff] at this
    use(⌊-⌊x⌋)*2
    norm_num
    apply@le_antisymm
    use not_lt.1 (by cases exists_nat_ge (1/(x-_)) with|_ N =>nlinarith[ one_div_mul_cancel $ sub_ne_zero.2
      ↪ ·.ne',9,Int.floor_le x, this N])
    use not_lt.1 (by cases exists_nat_ge (1/_:ℝ)with|_ A=>nlinarith[Int.lt_floor_add_one x,one_div_mul_cancel$
      ↪ sub_ne_zero.2 ·.ne',this A])
  intro
  induction‹_› using@Nat.strongInductionOn
  specialize L$ ‹_›+1
  simp_all[add_comm,mul_assoc,Int.floor_eq_iff,<-Nat.Ico_succ_right, add_mul,(Finset.range_succ),
    ↪ Finset.sum_Ico_eq_sum_range]
  revert‹ℕ›
  rintro A B@c
  simp_all[ Finset.mem_range.mp _,←eq_sub_iff_add_eq',Int.floor_eq_iff]
  suffices:∑d in .range A,⌊x+d*x⌋=∑Q in .range A,(⌊x⌋+2*(Q * (⌊-.floor x)))
  · suffices:∑d in ( .range A),(((d)):ℤ) =A * ( A-1)/2
    · have:(A : ℤ) * (A-1)%2=0
      · cases@Int.emod_two_eq A with|_ B=>norm_num[B,Int.sub_emod,Int.mul_emod]
      norm_num at*
      norm_num[ Finset.sum_add_distrib,<-Finset.sum_mul, ←Finset.mul_sum _ _] at*
      rw[eq_sub_iff_add_eq]at*
      zify[←mul_assoc, this,←eq_sub_iff_add_eq',‹_ =(@ _) /@_›,Int.floor_eq_iff] at *
      zify[*]at*
      cases S5:lt_or_ge c (A * (⌊-.floor ↑x)+⌊x⌋ + 1)
      · simp_all
        have:(c+1:ℝ)<=A*(⌊-⌊x⌋)+⌊x⌋+1:=by norm_cast
        simp_all
        cases this.eq_or_lt
        · repeat use by nlinarith
        nlinarith[(by norm_cast at* :(A*(⌊ -⌊x⌋):ℝ)+⌊(x)⌋ >=(c)+01),9,Int.add_emod ↑5,Int.floor_le (@x :
          ↪ ℝ),Int.lt_floor_add_one (x:)]
      simp_all
      nlinarith[(by norm_cast:(c:ℝ)>=A*(⌊-⌊_⌋)+⌊_⌋+1),Int.floor_le x,Int.lt_floor_add_one x]
    rw [←Nat.cast_sum, mul_sub, Finset.sum_range_id]
    cases A with|_=>norm_num[mul_add]
  use Finset.sum_congr rfl<|by simp_all[add_comm,Int.floor_eq_iff]
```